

Full Project Report & Documentation

Course	Deep Learning / Neural Networks
Platform	Synapse — Built from Scratch
Stack	Python · NumPy · Streamlit
Models	Perceptron · MLP · Regularization

2025 — 2026
Brahim Tennich – Mouhamed Meguebli

1.Introduction :

We were asked to develop an interactive educational platform that doesn't only implement neural networks but also teaches how they work. Thus we created "Synapse", an interactive educational platform built entirely from scratch in Python and NumPy.

It lets you configure, train, and visualize models in real time, from the classic Perceptron to full Multi-Layer networks, without hiding the math behind black-box libraries. Every weight, gradient, and activation is exposed so you actually understand what's happening, not just what the output says.

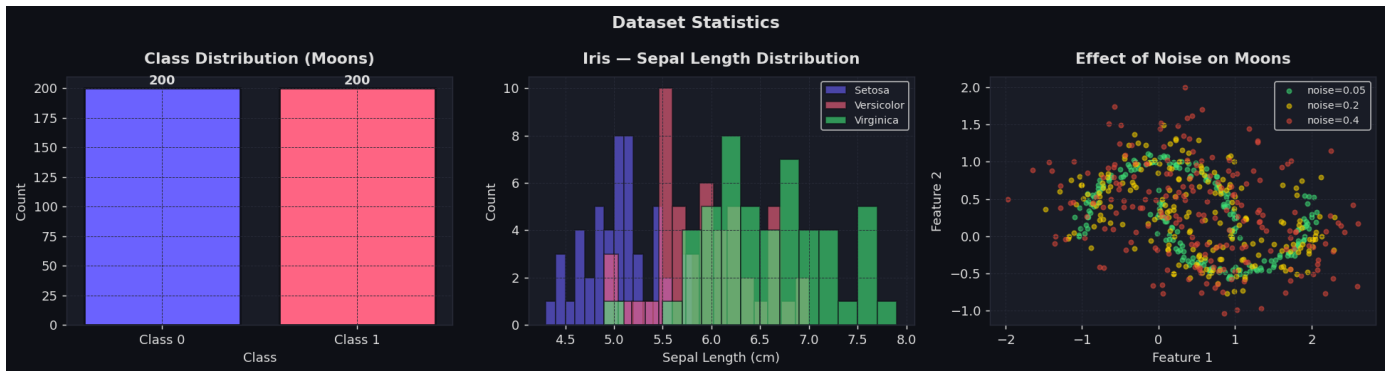
2.Platform Overview :

Synapse is built entirely from scratch using Python and NumPy, no ML libraries. The backend handles all model logic, the frontend is a Streamlit app that lets the user configure, train, and visualize everything in real time. The platform supports six built-in datasets covering binary classification, multi-class classification, and regression.

Figure 1 shows all six built-in datasets plotted in 2D feature space. Moons and Circles are clearly non-linear, no straight line can separate them. XOR has points from the same class scattered in opposite quadrants. Blobs and Iris are well-separated clusters. The Sine curve shows noisy samples around a smooth wave, used for regression.



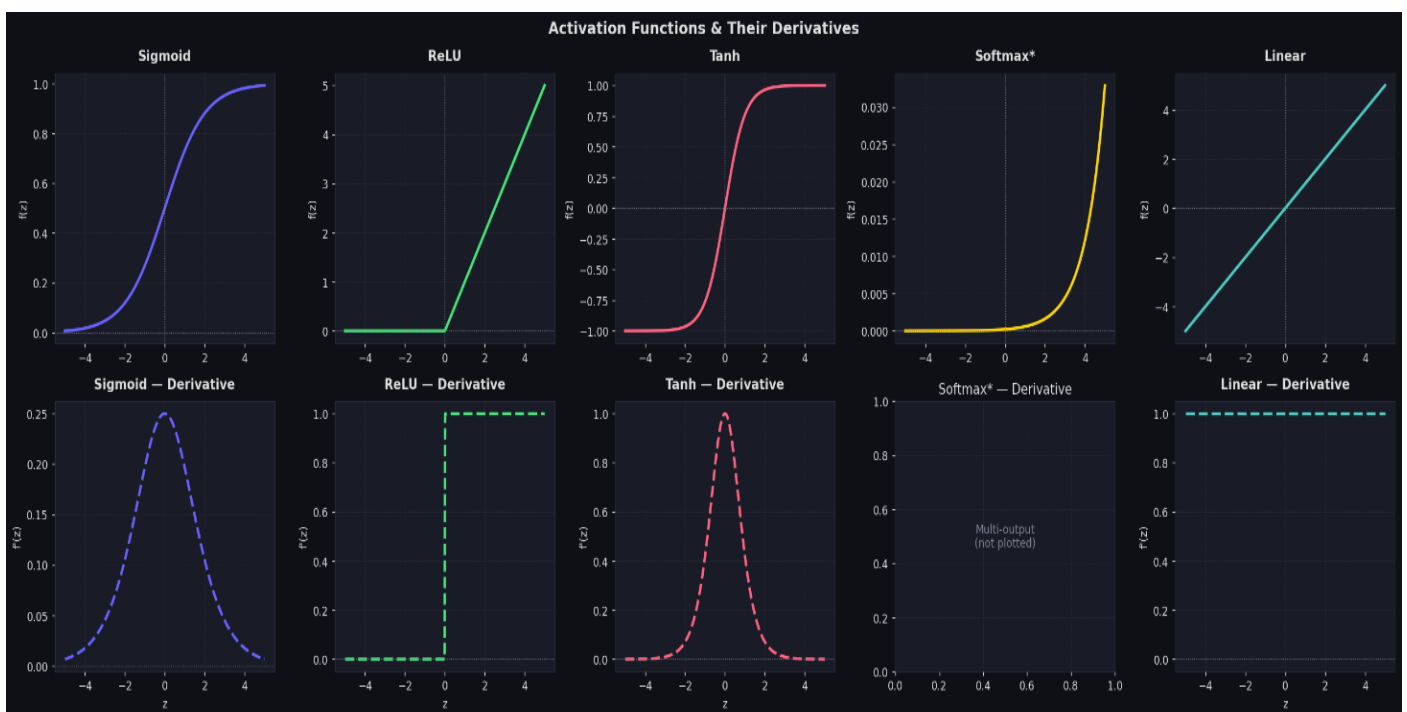
Figure 2 shows the class distributions are perfectly balanced (200 samples per class for Moons), and demonstrates how increasing noise progressively blurs the boundary between classes on the Moons dataset



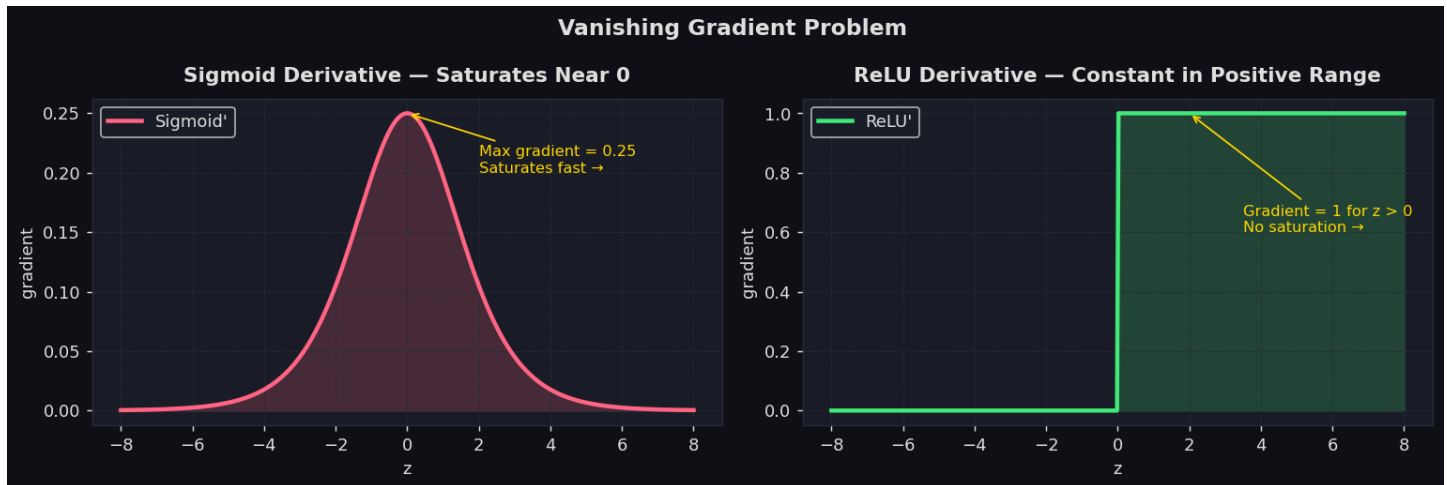
3.1.Theoretical Foundations :

Activation functions introduce non-linearity into the network. Without them, stacking multiple layers is mathematically equivalent to a single linear transformation. Synapse implements Sigmoid, ReLU, Tanh, Softmax, and Linear, each suited for different use cases. Sigmoid and Tanh work well for output layers, ReLU is the standard choice for hidden layers due to its simplicity and stability.

Figure 3 shows each activation function on the top row and its derivative on the bottom. Sigmoid produces an S-curve between 0 and 1. ReLU is flat at zero for negative inputs and linear for positive ones. Tanh is zero-centered, ranging from -1 to 1. The derivatives reveal why ReLU is preferred for hidden layers, it stays constant and non-zero for positive inputs, while Sigmoid and Tanh derivatives both shrink toward zero at the extremes



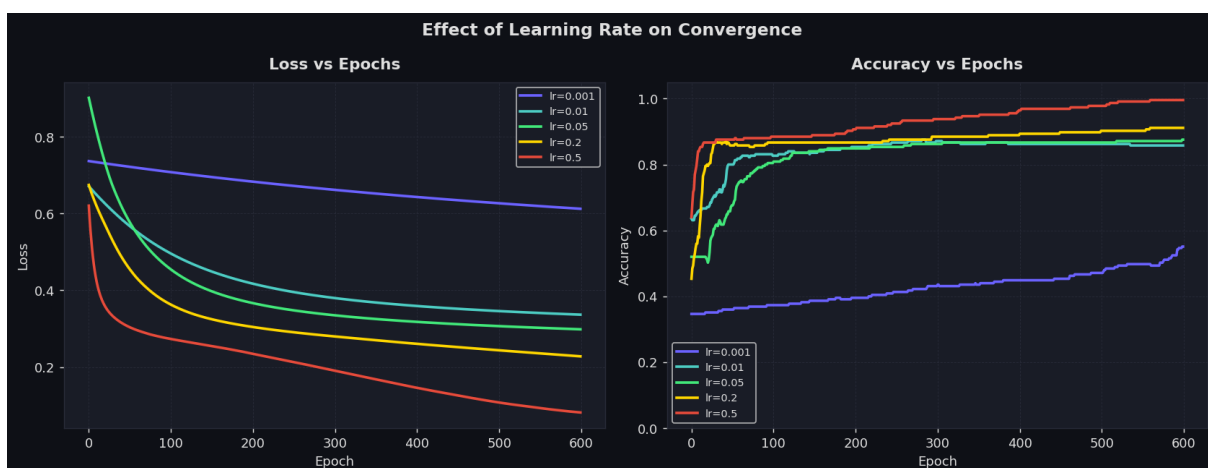
One important issue with Sigmoid is the vanishing gradient problem. Its derivative maxes out at 0.25 and approaches zero at the extremes. When gradients are multiplied across many layers during backpropagation, the signal shrinks to nearly nothing, early layers stop learning. ReLU fixes this: its derivative is a constant 1 for all positive inputs, so the gradient flows through unchanged.



The figure above shows each activation function on the top row and its derivative on the bottom. Sigmoid produces an S-curve between 0 and 1. ReLU is flat at zero for negative inputs and linear for positive ones. Tanh is zero-centered, ranging from -1 to 1. The derivatives reveal why ReLU is preferred for hidden layers, it stays constant and non-zero for positive inputs, while Sigmoid and Tanh derivatives both shrink toward zero at the extremes

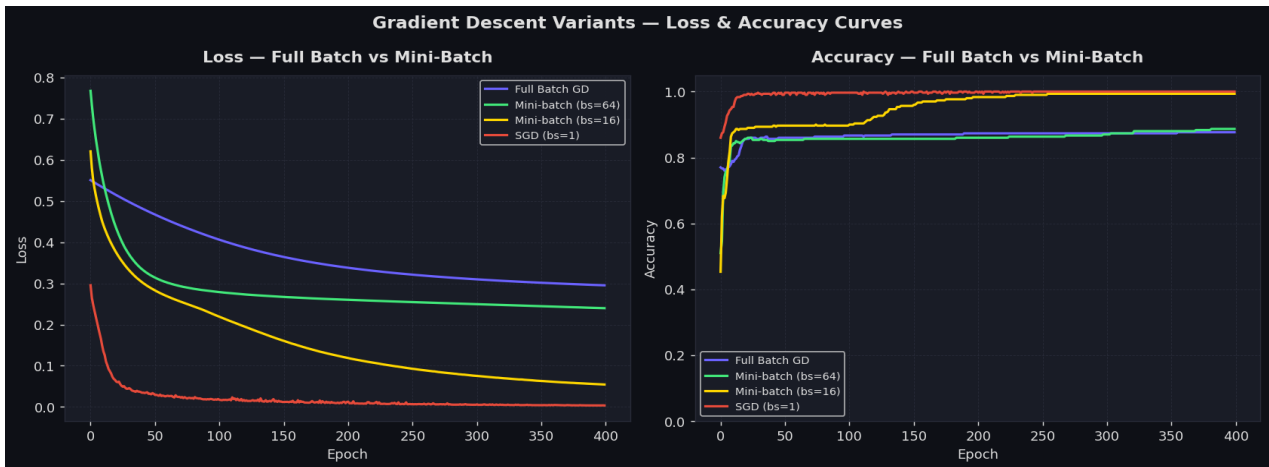
3.2 Gradient Descent and Learning Rate :

The learning rate controls how large each weight update is. Too small and the model barely moves, it takes hundreds of extra epochs to converge. Too large and it overshoots, oscillating around the minimum without ever settling.



The figure shows five learning rates tested on the same network. $lr=0.001$ barely converges even after 600 epochs, staying above 0.6 loss. $lr=0.5$ drops to near zero loss within the first 50 epochs and reaches 100% accuracy. The middle values show a smooth trade-off, faster convergence with higher learning rates, but at the cost of some stability.

Mini-batch gradient descent is a middle ground between full-batch and SGD. Full-batch uses all the data per update, smooth but slow. SGD updates after every single sample, fast but noisy. Mini-batch gets the best of both: faster convergence than full-batch, more stable than SGD.



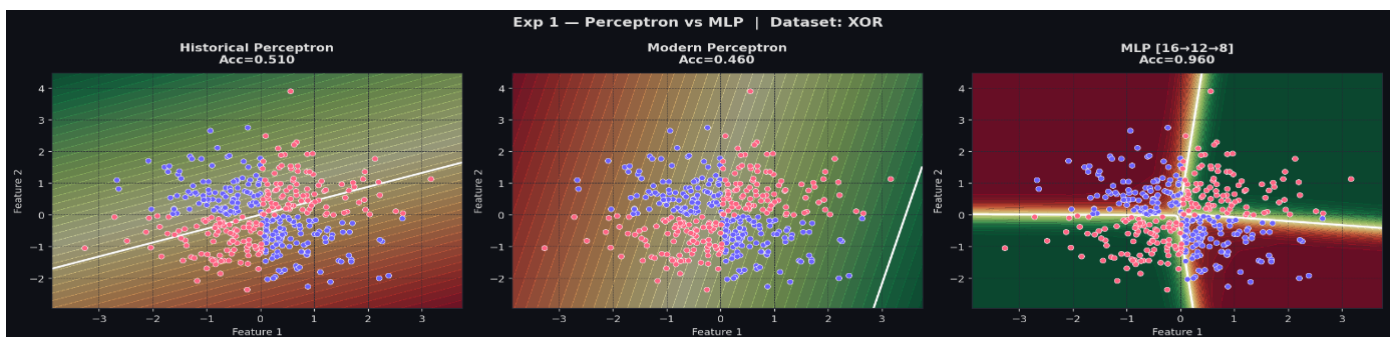
The figure shows this directly. The SGD curve (bs=1) in red drops to near-zero loss the fastest and hits 100% accuracy, but the curve is visibly noisy throughout. Full-batch in blue is the smoothest but converges the slowest, plateauing around 85% accuracy at epoch 400. Mini-batch with bs=16 hits the best balance, fast convergence and clean curves.

4. Experiment 1; Perceptron vs MLP :

The objective is to compare the Historical Perceptron, Modern Perceptron, and MLP on non-linear datasets.

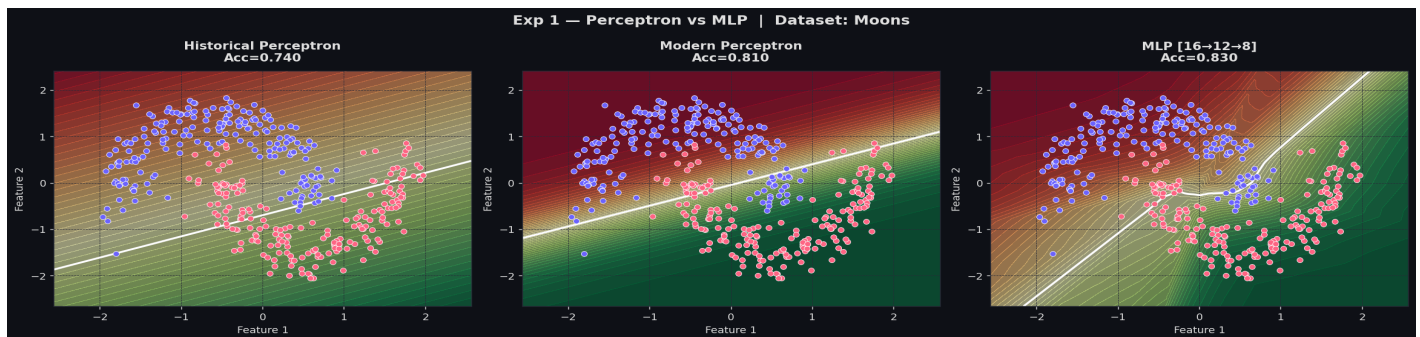
The core question here is simple: can a single decision boundary solve every problem? The answer is no. Perceptrons can only draw straight lines. On XOR, Moons, and Circles, all non-linear, they fail by design.

On XOR, both perceptrons scored around 50%, which is no better than random guessing. The MLP hit 96% by learning a cross-shaped boundary that a straight line physically cannot represent.

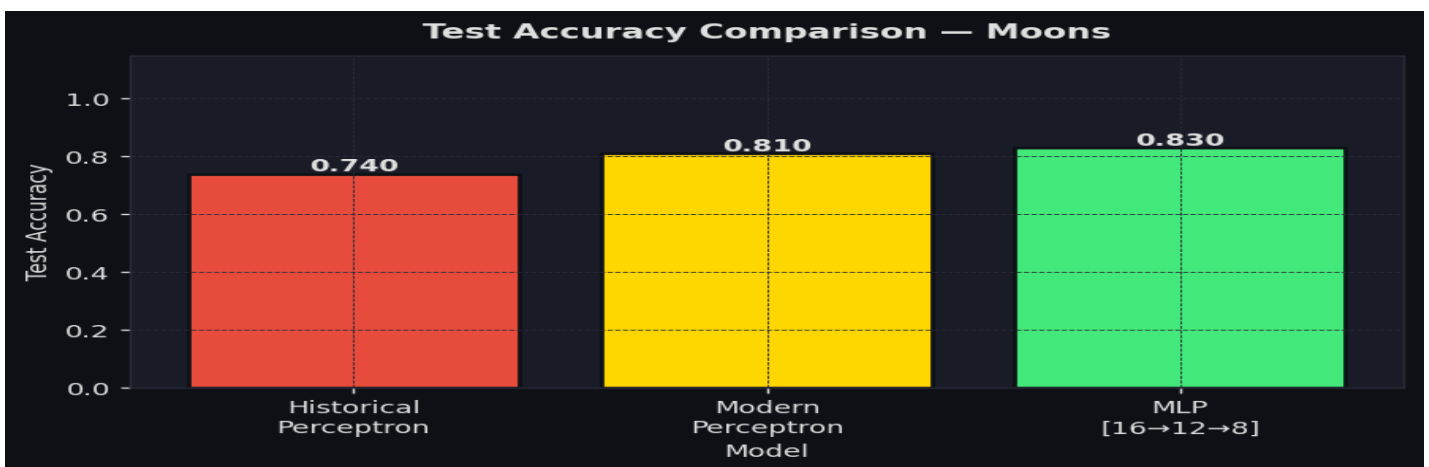


The figure shows this clearly, the perceptron draws a single diagonal line that cuts randomly through both classes, while the MLP produces two intersecting boundary lines that correctly isolate the XOR pattern.

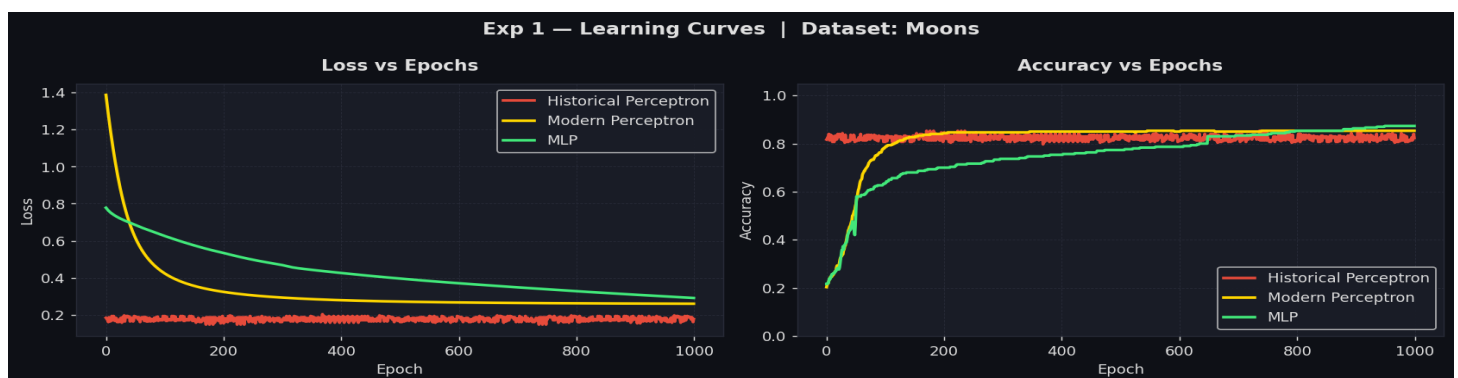
On Moons, the dataset is partially separable so the perceptrons managed 74% and 81%, but the MLP still pulled ahead at 83% with a curved boundary that actually follows the shape of the data. Figure 8 shows the Historical Perceptron drawing a straight line that misses the curved separation entirely, while the MLP boundary bends to follow the moon shapes.



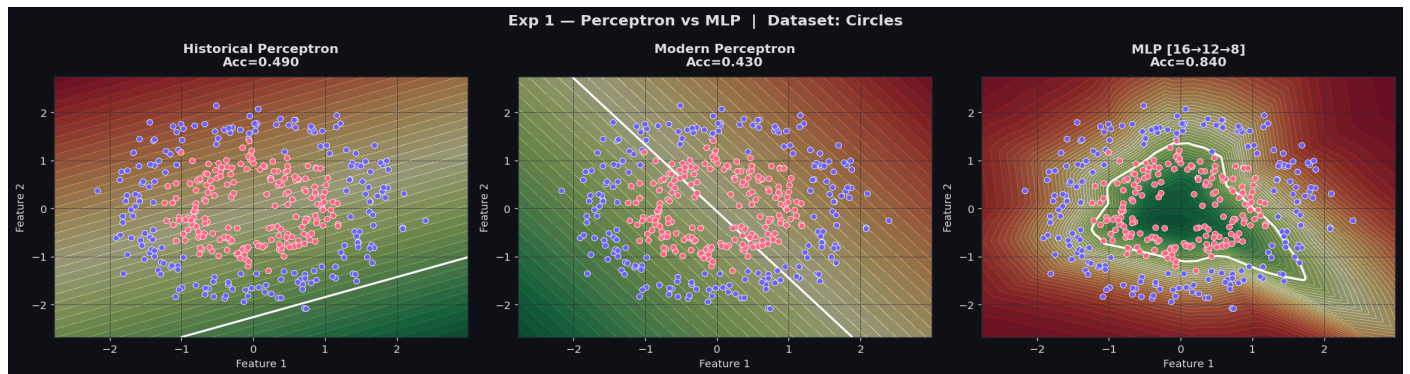
The figure below shows the accuracy bar chart, a clear step up from 0.74 to 0.81 to 0.83 as model complexity increases.



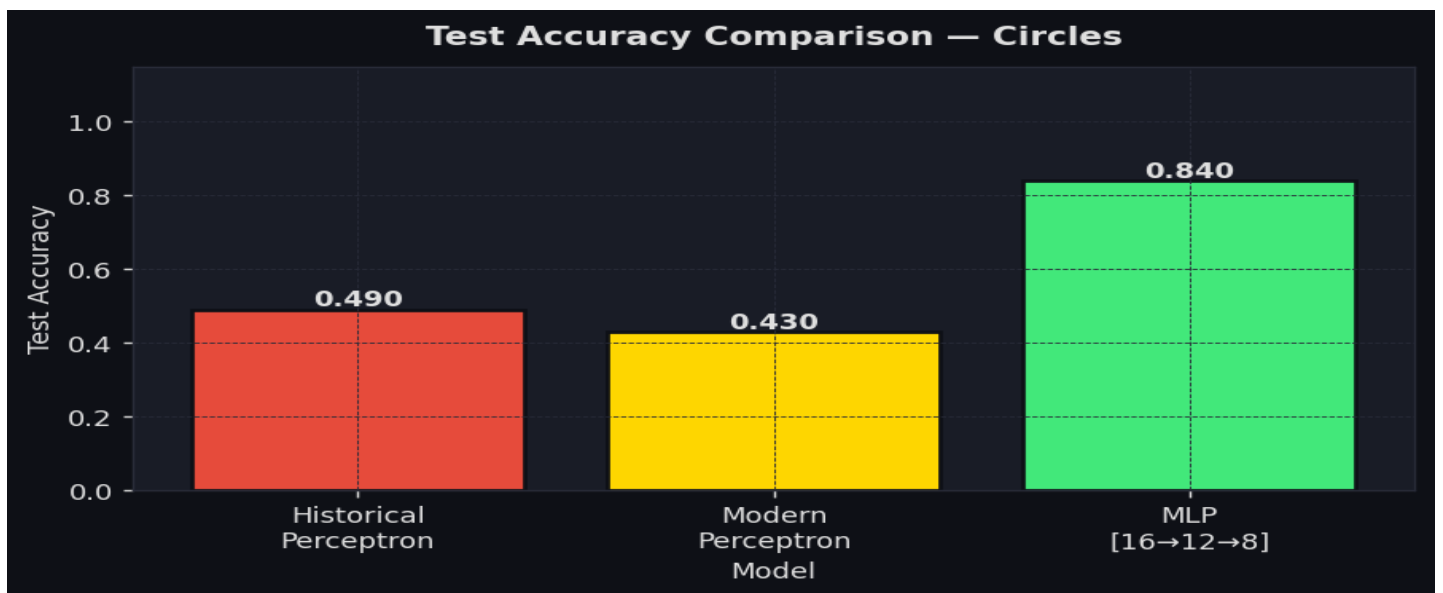
The figure shows the learning curves on Moons. The Historical Perceptron flatlines immediately since it doesn't use gradient descent. The Modern Perceptron converges smoothly. The MLP starts slower but eventually surpasses both.



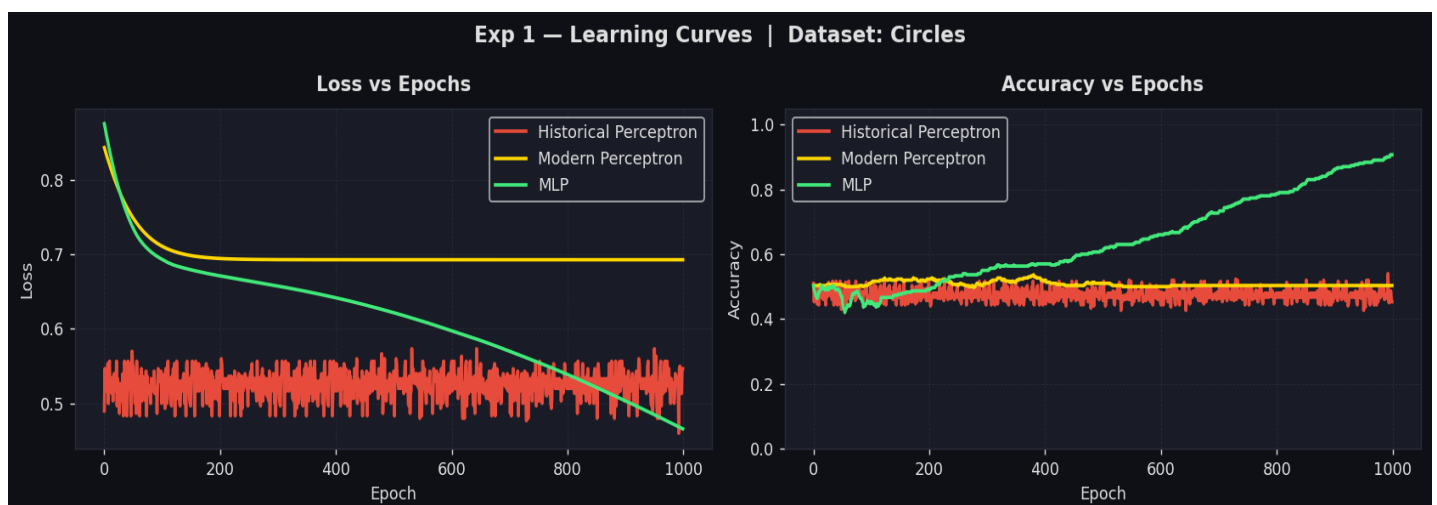
Circles was the most extreme case. Both perceptrons were essentially useless at 49% and 43%, the boundary needs to be circular, which a linear model cannot produce. Figure 11 shows the perceptron drawing a diagonal line through the middle of the dataset, completely ignoring the circular structure. The MLP reached 84% by wrapping a curved boundary around the inner cluster, which is visible as a closed loop in the decision boundary plot.



The figure below shows the accuracy gap, 0.49 and 0.43 for the perceptrons versus 0.84 for the MLP. The largest jump in the entire experiment.



The figure below shows the learning curves on Circles. Both perceptrons plateau at around 50% and never improve, they are structurally incapable of solving this problem. The MLP keeps climbing past 800 epochs and finishes well above both.



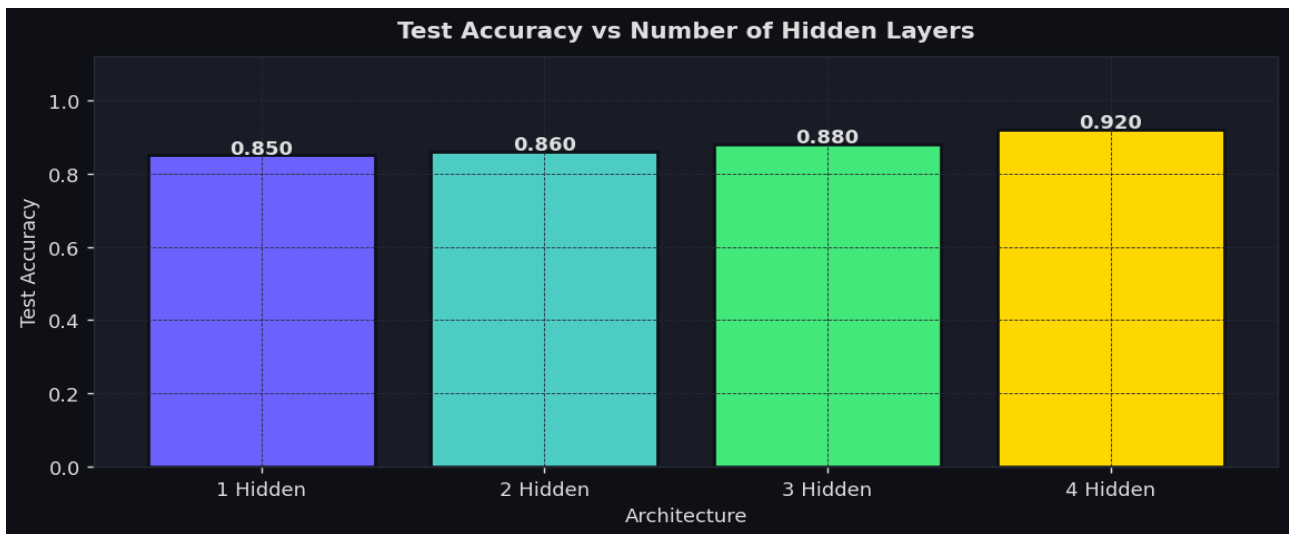
The takeaway is straightforward: as soon as the data isn't linearly separable, single-layer models hit a hard ceiling. The MLP doesn't just perform better, it's solving a fundamentally different problem.

5. Experiment 2 — Effect of Network Depth :

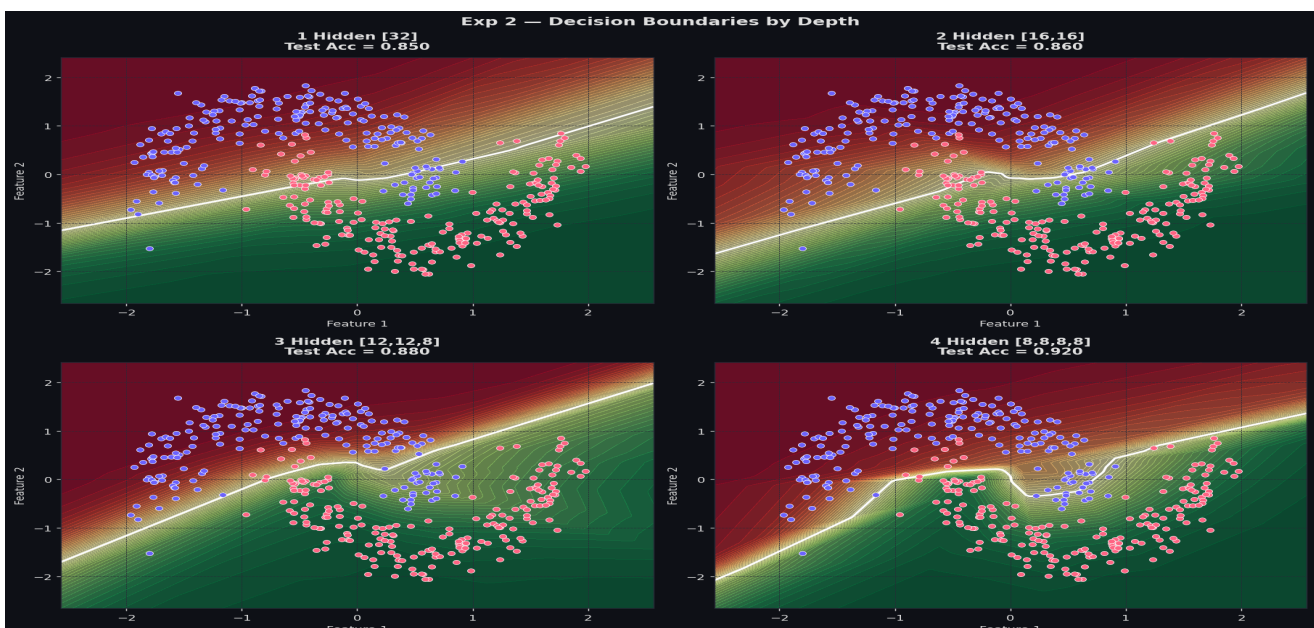
The objective is to check if adding more hidden layers improves performance.

We tested four architectures: 1 hidden layer [32], 2 hidden layers [16,16], 3 hidden layers [12,12,8], and 4 hidden layers [8,8,8,8], all trained for 1200 epochs on the same dataset.

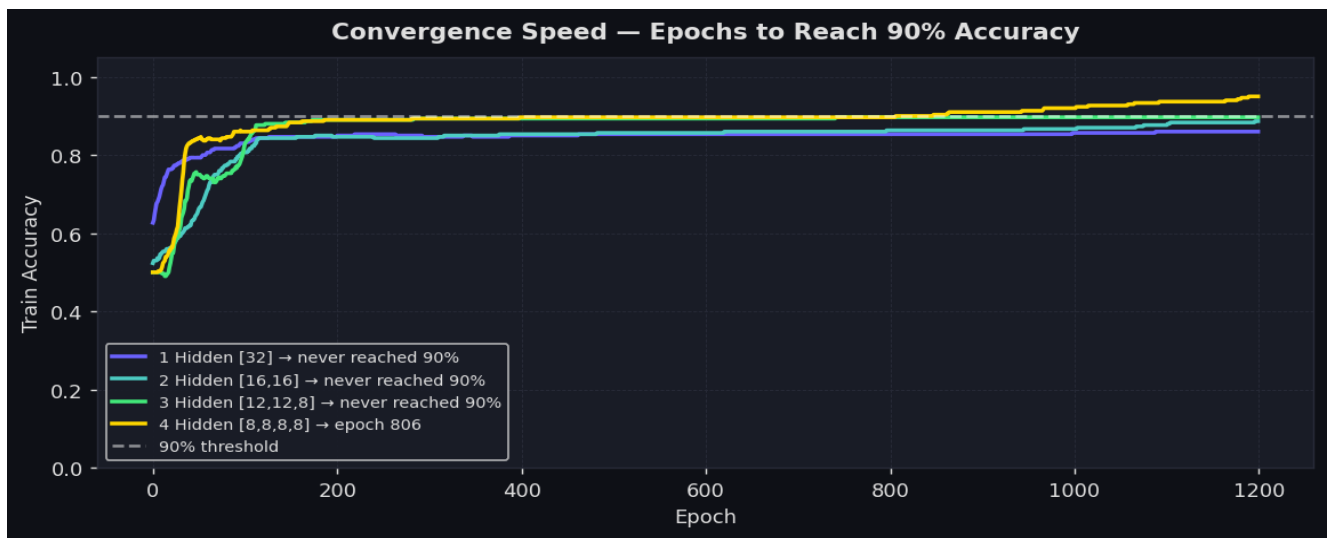
Accuracy improved consistently with depth: 85%, 86%, 88%, and 92%. Figure 14 shows this as a clean staircase bar chart, each additional layer adds meaningful improvement, with the biggest jump going from 3 to 4 hidden layers.



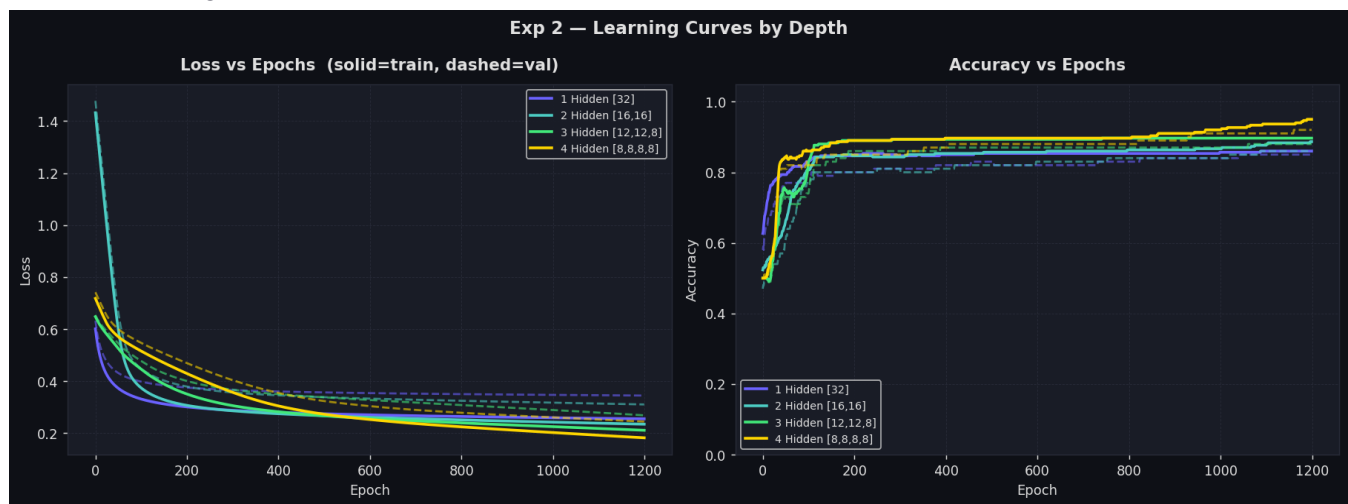
The figure shows the decision boundaries for all four architectures side by side. The 1-hidden-layer network draws a relatively straight boundary that misclassifies a visible chunk of points. As depth increases, the boundary becomes more curved and precise, with the 4-layer network producing the cleanest separation.



The figure shows convergence speed. Only the 4-hidden-layer network crossed the 90% accuracy threshold, reaching it at epoch 806. The shallower networks plateau below 90% and never cross the dashed line despite training for 1200 epochs.



The figure shows the full loss and accuracy curves. All architectures converge, but deeper networks achieve lower final loss. The train and validation curves stay close across all architectures, meaning the added depth improved generalization rather than causing overfitting.

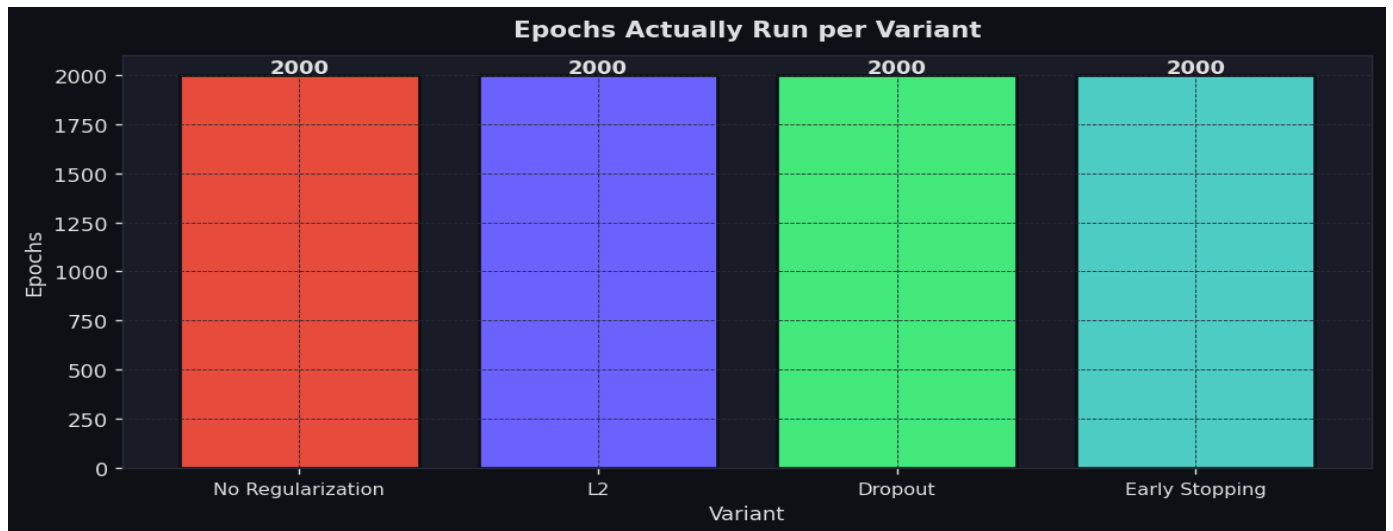


The takeaway is that More layers means better accuracy and more expressive decision boundaries, but it comes at the cost of convergence speed. The 4-hidden-layer network was the only one to cross 90% accuracy, but it took until epoch 806 to get there. Shallower networks learn faster but plateau at a lower ceiling. The good news is that none of the architectures showed signs of overfitting, train and validation curves stayed close throughout, so the depth was genuinely helping the model learn better representations, not just memorizing training data.

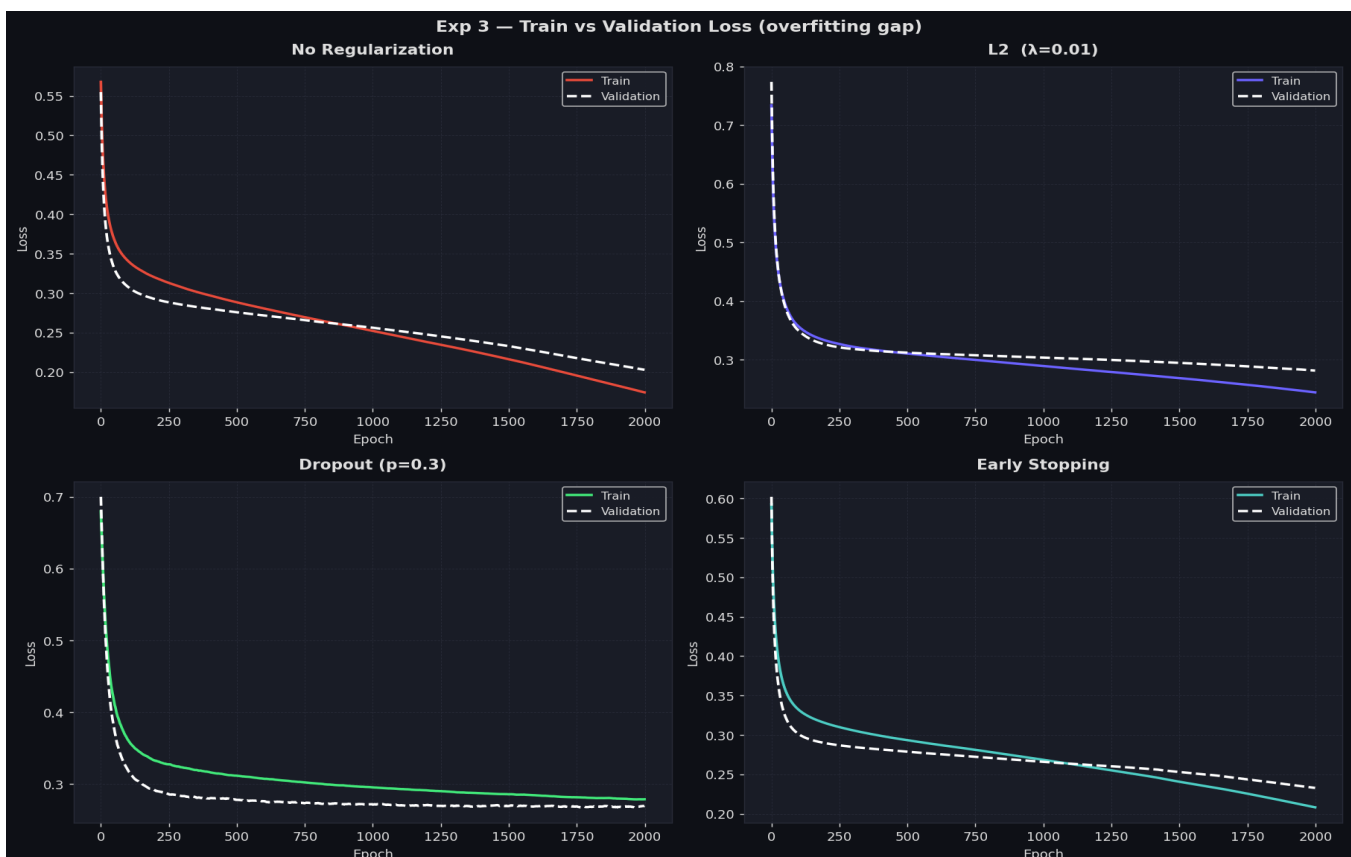
6. Experiment 3 : Overfitting vs Regularization :

The objective is to compare: No Regularization, L2, Dropout, and Early Stopping on the same network.

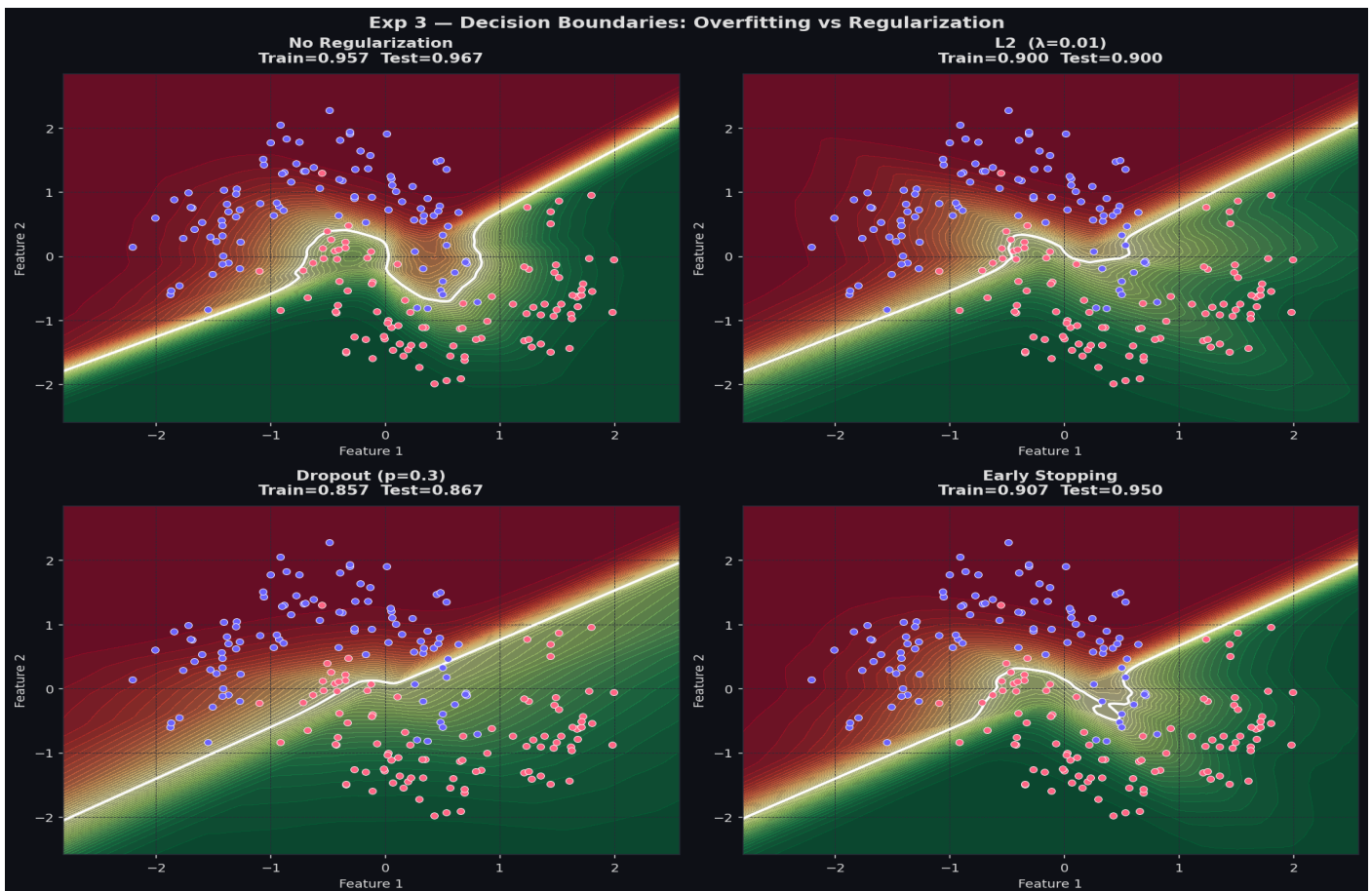
All four variants were trained for 2000 epochs as shown in Figure 18, none terminated early, meaning the dataset was clean enough that early stopping never triggered during training.



The figure shows the train vs validation loss curves for all four variants. The unregularized model shows the widest gap, training loss keeps dropping while validation loss flattens, a textbook overfitting pattern. L2 keeps the two curves much closer by penalizing large weights. Dropout and Early Stopping both reduce the gap through different mechanisms, Dropout by randomly disabling neurons each step, Early Stopping by halting before the model overfits.



The figure shows the decision boundaries for all four variants. The unregularized boundary is visibly jagged and over-fitted, bending around individual training points. L2 produces a smoother, straighter boundary. Dropout and Early Stopping produce boundaries that are curved but not erratic, they follow the general pattern of the data without chasing noise.



The figure below summarizes the overfitting gap, the difference between train and test accuracy. The unregularized model and Dropout both show a small negative gap (-0.010), meaning test accuracy was slightly higher than train. Early Stopping had the largest gap at -0.043, meaning its test accuracy (95%) significantly exceeded its train accuracy (90.7%), a strong sign that it generalized well and stopped before memorizing.



The main conclusion from this experiment is that raw accuracy alone doesn't tell the whole story. The unregularized model looked the best on paper at 95.7% train accuracy, but its boundary was the least trustworthy. Regularization trades a bit of training accuracy for a model that actually generalizes.

7. Conclusion :

Building Synapse from scratch made one thing clear, understanding neural networks isn't just about getting good accuracy numbers. Every experiment revealed a layer of nuance that a black-box library would have hidden. Perceptrons aren't broken, they're just limited by design. Depth isn't always better, it's a trade-off between capacity and convergence. Regularization isn't about fixing overfitting after the fact, it's about building models that generalize from the start.

The platform itself proved the point, being able to watch the decision boundary shift in real time, see the loss curves diverge, and control every hyperparameter manually is what turns theory into actual understanding. That was the whole idea behind building it this way.